

¡Por supuesto, Sandro! Es una excelente idea dejar esto respaldado en un documento limpio y ordenado para que no se pierda nada de lo que hemos conversado y lo puedas revisar cada vez que te sientes a programar.

Aquí tienes la estructura completa del proyecto en formato de documento técnico. Puedes copiar este texto directamente, pegarlo en un archivo de Word (o Google Docs) y guardarlo como PDF.

GUÍA DE IMPLEMENTACIÓN: SISTEMA GEO-ATMÓSFERICO DE DETECCIÓN SÍSMICA (PROTOTIPO CHILE)

Autor: Sandro **Enfoque:** Integración de Datos Satelitales, Meteorológicos y Sismológicos en Tiempo Real **Tecnología Base:** Python + Streamlit + Google Earth Engine (GEE)

1. INTRODUCCIÓN Y FUNDAMENTO CIENTÍFICO

El objetivo de este sistema es romper el aislamiento entre las distintas disciplinas científicas (Geología, Meteorología y Astrofísica) creando un **software integrador**. El sistema busca correlacionar la deformación física de la corteza con anomalías en la baja y alta atmósfera (Acoplamiento Litósfera-Atmósfera-Ionósfera) para identificar puntos críticos de tensión antes de una ruptura tectónica.

Las 3 Variables Clave:

1. **Deformación Terrestre (InSAR):** Movimientos milimétricos del suelo detectados por radar (Satélite Sentinel-1). Indica dónde la falla está "trabada" acumulando energía.
2. **Anomalías Térmicas Marinas (SST):** Liberación de gases y fricción microscópica que elevan la temperatura de la capa de aire y agua sobre la Fosa de Atacama (Satélites MODIS/NOAA).
3. **Sismicidad de Fondo (Ondas P / Enjambres):** Actividad de micro-sismos (Magnitud 2.0 a 3.5) que actúan como precursores mecánicos en la zona bloqueada.

2. ARQUITECTURA GENERAL DEL SOFTWARE

El sistema opera como un pipeline de datos cerrado de cuatro etapas principales:

```
[ INGESTA DE DATOS ]  —>  [ FILTRADO Y RUIDO ]  —>  [ MOTOR  
MATEMÁTICO ]  —>  [ INTERFAZ / MAPA ]  
- API USGS (Sismos)           - Remover clima (ERA5)           - Matriz de  
Decisión                     - Dashboard Web  
- Copernicus (InSAR)         - Filtrar tormentas           - Cálculo de  
Score                       - Alertas Visuales  
- NASA MODIS (Térmico)       solares (Ionósfera)           (0 a 100)
```

3. FASE 1: ENTORNO DE DESARROLLO E INSTALACIÓN

Para comenzar a ejecutar el proyecto de a poco, necesitas preparar tu entorno de programación local en Python.

Requisitos del Sistema:

- Python 3.9 o superior instalado.
- Consola de comandos (Terminal / CMD) con acceso a pip.

Comando de Instalación de Librerías:

Ejecuta el siguiente comando en tu terminal para instalar los componentes de la interfaz, manejo de datos y mapas interactivos:

```
pip install streamlit pandas requests folium streamlit-folium
```

4. FASE 2: CÓDIGO FUENTE DEL PROTOTIPO (app.py)

Guarda el siguiente código en un archivo llamado app.py. Este archivo contiene el esqueleto del software, la conexión real a los sismos de Chile y los controles simulados para calibrar el algoritmo.

```
import streamlit as st
import pandas as pd
import requests
import folium
from streamlit_folium import st_folium
from datetime import datetime, timedelta

# CONFIGURACIÓN DE LA INTERFAZ WEB
st.set_page_config(page_title="Sistema Tectónico Geo-Atmosférico",
layout="wide")

st.title("🌐 Sistema Integrado de Monitoreo Sísmico y Atmosférico")
st.subheader("Prototipo de Detección de Anomalías y Puntos de Tensión en la Fosa de Atacama")
st.markdown("---")

# PANEL LATERAL: CONTROLES DE AJUSTE (SIMULACIÓN DE SATÉLITES)
st.sidebar.header("🔧 Parámetros del Sistema")

# Sliders para calibrar el comportamiento del algoritmo antes de
```

```

conectar las API espaciales
insar_tension = st.sidebar.slider("Nivel de Bloqueo de Falla (InSAR
%)", 0, 100, 75)
thermal_anomaly = st.sidebar.slider("Anomalía Térmica Marina (SST °C
sobre el promedio)", 0.0, 5.0, 2.8, step=0.1)

st.sidebar.markdown("---")
st.sidebar.info("""
**Fuentes de Datos Integradas:**
-  Sismicidad: USGS API (Tiempo Real)
-  Deformación: Sentinel-1 InSAR (Simulado en Slider)
-  Térmico: MODIS/NOAA SST (Simulado en Slider)
""")

# MOTOR DE CORRELACIÓN Y PONDERACIÓN MATEMÁTICA
def calcular_riesgo(insar, termico, total_sismos):
    # Fórmula base: 40% InSAR, 40% Térmico, 20% actividad sísmica
    local acumulada
    score = (insar * 0.4) + (termico * 10 * 0.4) + (min(total_sismos,
10) * 2)

    if score >= 75:
        return "CRÍTICO / PUNTO DE RUPTURA POTENCIAL", "", score
    elif score >= 45:
        return "ATENCIÓN / TENSION EN ACUMULACIÓN", "", score
    else:
        return "ESTABLE / RUIDO BASE", "", score

# CONEXIÓN REAL A LA API DEL USGS (CUADRANTE DE CHILE)
@st.cache_data(ttl=300) # Caché de 5 minutos para optimizar el
rendimiento
def obtener_sismos_chile():
    # Consulta los sismos ocurridos en el cuadrante de Chile en los
últimos 7 días
    fecha_inicio = (datetime.now() -
timedelta(days=7)).strftime('%Y-%m-%d')
    url =
f"https://earthquake.usgs.gov/fdsnws/event/1/query?format=geojson&star
ttime={fecha_inicio}&minlatitude=-56&maxlatitude=-18&minlongitude=-76&
maxlongitude=-66"

    try:
        response = requests.get(url)
        data = response.json()

        sismos = []
        for feature in data['features']:
            coords = feature['geometry']['coordinates']

```

```

        props = feature['properties']
        sismos.append({
            "Magnitud": props['mag'],
            "Lugar": props['place'],
            "Latitud": coords[1],
            "Longitud": coords[0],
            "Profundidad (km)": coords[2],
            "Fecha":
datetime.fromtimestamp(props['time']/1000).strftime('%Y-%m-%d %H:%M')
        })
    return pd.DataFrame(sismos)
except Exception as e:
    st.error(f"Error al conectar con la API de sismos: {e}")
    return pd.DataFrame()

df_sismos = obtener_sismos_chile()
total_sismos_recientes = len(df_sismos) if not df_sismos.empty else 0

# Procesar las variables en el motor de decisión
estado, icono, puntaje = calcular_riesgo(insar_tension,
thermal_anomaly, total_sismos_recientes)

# VISUALIZACIÓN EN EL DASHBOARD (MÉTRICAS PRINCIPALES)
col1, col2, col3 = st.columns(3)
col1.metric(label="Estado del Sistema", value=f"{icono}
{estado.split('/')[0]}")
col2.metric(label="Índice de Tensión Calculado", value=f"{puntaje:.1f}
/ 100")
col3.metric(label="Micro-sismos Detectados (7 días)",
value=total_sismos_recientes)

st.markdown("---")

# DISTRIBUCIÓN DE LA INTERFAZ (MAPA E HISTORIAL)
col_mapa, col_datos = st.columns([2, 1])

with col_mapa:
    st.subheader("🌐 Mapa de Monitoreo Dinámico")

    # Centrar el mapa en la zona de subducción (Chile Central)
    m = folium.Map(location=[-33.0, -71.5], zoom_start=6,
control_scale=True)

    # Dibujar la Fosa de Atacama en el océano (Zonas de profundidad
extrema frente a la costa)
    fosa_coords = [[-20, -71.5], [-25, -71.5], [-30, -72.0], [-35,
-73.0], [-40, -74.5]]
    folium.PolyLine(fosa_coords, color="blue", weight=3, opacity=0.7,

```

```

        tooltip="Eje aproximado de la Fosa de Atacama
(Profundidad >6000m)").add_to(m)

# Proyección del Punto de Ruptura si los indicadores superan el
umbral crítico
if puntaje >= 75:
    folium.CircleMarker(
        location=[-33.0, -72.2], # Punto crítico hipotético en la
costa de la V región
        radius=25,
        color="red",
        fill=True,
        fill_color="red",
        fill_opacity=0.4,
        popup="⚠️ ALERTA: Concentración crítica de anomalía
térmica + deformación mecánica"
    ).add_to(m)

# Desplegar los sismos reales de la API en el mapa
if not df_sismos.empty:
    for idx, row in df_sismos.iterrows():
        folium.CircleMarker(
            location=[row['Latitud'], row['Longitud']],
            radius=float(row['Magnitud']) * 2,
            color="orange" if row['Magnitud'] < 5 else "red",
            fill=True,
            popup=f"Mag: {row['Magnitud']} - {row['Lugar']}
({row['Fecha']})"
        ).add_to(m)

# Renderizar mapa
st_folium(m, width=800, height=500)

with col_datos:
    st.subheader("📋 Últimos Eventos Registrados")
    if not df_sismos.empty:
        st.dataframe(df_sismos[['Magnitud', 'Fecha', 'Profundidad
(km)']].sort_values(by="Magnitud", ascending=False), height=450)
    else:
        st.write("No se encontraron sismos recientes.")

```

5. FASE 3: EJECUCIÓN Y PRUEBAS DEL SISTEMA

1. Guarda el código anterior estrictamente con el nombre `app.py`.
2. Abre tu terminal de comandos, navega hasta la carpeta del archivo y ejecuta:
`streamlit run app.py`

3. El sistema levantará un servidor local y abrirá una ventana en tu navegador (<http://localhost:8501>).
4. **Prueba de estrés del algoritmo:** Mueve los deslizadores de la barra lateral al máximo para verificar cómo el motor gráfico proyecta el círculo rojo de "**Punto de Ruptura Potencial**" en la costa chilena frente a la Región de Valparaíso, simulando el cruce crítico de datos.

6. HOJA DE RUTA PARA FUTURAS ACTUALIZACIONES (DE ANALÓGICO A AUTOMÁTICO)

Para pasar este prototipo a un nivel profesional e industrial, los pasos a seguir de forma progresiva son:

- **Hito 1 (Corto Plazo):** Reemplazar el slider térmico conectando el script a la API de **Google Earth Engine (GEE)** para descargar en tiempo real el producto MODIS/061/MOD11A1 (Temperatura de la Superficie Terrestre y Marítima).
- **Hito 2 (Medio Plazo):** Integrar datos de procesamiento automático InSAR utilizando los repositorios públicos de la infraestructura **Copernicus (ESA)** para monitorquear las deformaciones de la cordillera de la Costa.
- **Hito 3 (Largo Plazo):** Incorporar un modelo de Machine Learning supervisado (redes neuronales recurrentes LSTM) entrenado con los datos de las anomalías previas a los grandes terremotos históricos de Chile (2010, 2014, 2015).

¡Listo! Con esto tienes un documento técnico impecable para guardar. Avísame cuando lo tengas copiado y si necesitas que en el futuro empecemos a picar el código para el Hito 1 (conectar Google Earth Engine).